Proyecto DevOps

El objetivo del obligatorio es la aplicación de prácticas de ingeniería de software ágil y DevOps para mejorar la calidad de una aplicación. Se implementarán prácticas de CI/CD, QA y testing ágil sobre una base de código existente. El proyecto que se usará como base utiliza lenguajes y tecnologías del semestre anterior de la carrera.

En el informe final del proyecto se debe evaluar la mejora en la calidad del software y la performance del equipo usando métricas de DevOps. Se debe reflexionar sobre el impacto de las prácticas de ingeniería de software ágil y DevOps.

Actividades

- Aplicar un marco de gestión ágil
- Analizar la deuda técnica
- Implementar un repositorio y procedimientos de versionado
- Crear un pipeline de CI/CD e integrar prácticas de QA en el pipeline
- Generar escenarios de testing desde la perspectiva del usuario
- Aplicar técnicas de testing ágil y gestionar el feedback
- Agregar telemetría a la aplicación
- Analizar la performance utilizando métricas de DevOps
- Reflexionar sobre el impacto de las prácticas de ingeniería de software ágil

Entregas y esfuerzo

La ejecución del proyecto tiene 5 entregas de duración variable entre dos y tres semanas, con la confección final de un informe académico que resuma los resultados y aprendizajes del proyecto.

El proyecto se realizará en el marco de gestión Kanban. Se espera que cada integrante dedique un mínimo de 5 horas semanales a las actividades de ingeniería del proyecto (sin incluir gestión y retrospectivas). Antes de cada entrega, el equipo realizará una retrospectiva y confeccionará un informe de avance. La retrospectiva deberá considerar algún método conocido (por ej. DAKI) y su análisis debe estar basado en datos.

En cada informe de avance y en el informe académico final se deben detallar las actividades realizadas y justificar las decisiones tomadas para cumplir con los objetivos del proyecto.

Producto a construir/modificar

En su oportunidad el equipo docente hará entrega del producto de software (ya construido) que cada uno de los equipos deberán mantener, ya sea por reparación de bugs o extensión de funcionalidades. Las nuevas funcionalidades y mejoras para desarrollar serán comunicadas oportunamente por Aulas.

Cada entrega implica realizar actividades sobre la base de código provista por los docentes. La evolución de la solución de software debe ser realizada en GitHub. Las automatizaciones deben realizarse en GitHub Actions y la gestión de issues en GitHub Projects.

Informes de avance

En cada entrega el informe de avance debe incluir.

- Registro de actividades realizadas (fecha, actividad, horas, integrante), cada actividad debe tener al menos un commit en el repositorio realizado en la fecha reportada.
- Evidencia de resultados de la entrega (detallados en la rúbrica de evaluación)
- Referencias a cualquier documento que respalde actividades particulares. Se puede pensar que el informe debe incluir un índice hacia cualquier otro documento relevante para la entrega actual.
- Mejoras en la calidad y evidencia de revisión con el PO (uno de los integrantes debe tomar el rol en cada etapa)
- Retrospectiva y acciones de mejora

Nota: el marco Kanban y el proceso de ingeniería se definen en la primera entrega. En las entregas posteriores **se debe reportar si se realizaron cambios en alguno de los procesos** de gestión o ingeniería, o si no hubo cambios.

Los informes de avance deben tener una extensión máxima de 5 páginas. El informe académico final puede tener una extensión de 10 páginas. Los resultados detallados de cada etapa pueden ser anexados con links al repositorio GitHub.

Se puede utilizar herramientas de IA para las actividades de ingeniería, documentando su uso (herramienta de IA, prompts, resultados obtenidos, reflexión sobre el impacto en la calidad). No se admite el uso de herramientas de IA para realizar los informes de avance y el informe académico final.

Procedimientos de entrega:

1. En el cierre de cada etapa se debe realizar merge a main en el repositorio GitHub y agregar un tag a la versión: 'Entrega N'.
2. El informe de avance de las primeras 4 etapas, se entregará subiendo un archivo PDF a Aulas. Se debe entregar al terminar cada etapa del proyecto hasta las 21 hs. del día indicado. Si el día de entrega coincide con un feriado no laborable, la misma puede prorrogarse hasta un día no feriado.
3. El informe académico final (quinta etapa) debe entregarse por el sistema de gestión de ORT hasta las 21 hs. del día que figura en Gestión. Los estudiantes deben estar inscriptos y formar equipo de obligatorio en el sistema. Si no se cumple con los requisitos administrativos de la universidad no se puede corregir el obligatorio.

Rúbrica de evaluación

Duración	Actividades requeridas	Puntos
2 sem	• Definición de marco de gestión basado en Kanban. Roles y ceremonias. Todos los miembros del equipo deben ser desarrollador y tester en todas las entregas. Los demás roles deben rotar. • Definición de un primer proceso de ingeniería. • Configuración del repositorio en GitHub: archivos y directorios, estrategia inicial de branching, estándares de nomenclatura.	5

	• Análisis de deuda técnica y gestión de la calidad: definición de un modelo de calidad y un análisis de deuda técnica en base a ese modelo de calidad. • Definición de issues y clasificación.	
2 sem	• Configuración del pipeline CI/CD en función del marco de gestión y procesos de ingeniería. Documentar pipeline con pasos automáticos y manuales. • Elección de dos bugs registrados como issues en GitHub, fundamentando la elección. Los bugs deben ser de la mayor severidad. Al menos uno debe ser funcional. • Reparación de ambos bugs utilizando TDD en el backend. Se deben rotular los commits en etapas [Green] y [Refactor]. No debe haber actualizaciones de código en el servidor sin actualización de pruebas que lo respalden.	6
3 sem	• Adaptación de los procesos de gestión e ingeniería para aplicar BDD. • Escritura de US, criterios de aceptación y escenarios de prueba BDD para las nuevas funcionalidades. El comportamiento debe especificarse en forma colaborativa: PO, tester, desarrollador. • Implementación de nuevas funcionalidades. Primero se debe implementar en el front-end con BDD. Luego en el back-end con TDD. Para ambos marcos de trabajo deben rotularse todos los commits para las nuevas funcionalidades con [TDD-GREEN], [TDD-REFACTOR] para el back y para el front, y [BDD-GREEN], [BDD-REFACTOR] para el front. • Vista de procesos 4+1 para las dos funcionalidades nuevas. Diagrama de componentes para **toda** la solución.	8
2 sem	• Implementación de un nuevo tipo de testing en el proyecto trabajado en el curso. Justificar relevancia del tipo de testing de acuerdo al modelo de calidad y mostrar evidencia de su realización. • Cálculo, justificación e interpretación de métricas DevOps a elección que hagan sentido en el contexto del proyecto. • Implementación de telemetría. El sistema debe poder emitir logs, 2 métricas de negocio, 2 métricas a nivel de aplicación y 2 métricas a nivel de infraestructura. Todas ellas se deben justificar y reportar su utilidad. Las métricas deben registrarse en un colector de telemetría en Prometheus usando OTLP. Los logs pueden guardarse en texto plano o en la misma base de datos del sistema.	7
2 sem	• Informe académico final. Al final de este documento se detalla el contenido esperado.	4

Tecnología y herramientas

- Tableros, repositorios y pipelines CI/CD: GitHub
- BDD: Cypress + Cucumber
- Desarrollo: .NET / Angular

- Grafana, OTLP y Prometheus.

Se espera que el equipo investigue y aplique técnicas y herramientas sobre el pipeline que permitan mejorar el valor entregado al cliente, en el contexto tecnológico específico del proyecto, tales como:

- Análisis estático de código y linters
- Ejecución automática de tests unitarios
- Análisis de cobertura de código
- Revisión de código mediante pull request
- En forma opcional, se pueden aplicar en el proyecto estrategias de despliegue en la nube y otras herramientas de QA (testing de performance, testing de seguridad, etc.)

Incluir estos elementos de extensión pueden ayudar a compensar otras áreas débiles del proyecto.

Guía para la realización del informe académico final

El informe académico final no debe contener detalles de implementación técnica. Debe ser una síntesis global del proyecto.

1. Resumen de gestión del proyecto

Incluir un resumen de la gestión de proyecto que agrupe las cinco etapas del proyecto. Detallar: actividades realizadas, entregables de cada actividad, cantidad de issues reportadas y cerradas, esfuerzo y métricas del proyecto.

2. Evolución de la calidad

De acuerdo con el modelo de calidad establecido, explicar la evolución de la calidad a lo largo del proyecto. Presentar evidencia de mejora en atributos de calidad específicos y realizar una evaluación global de calidad al momento de cierre del proyecto.

3. Reflexiones sobre las prácticas de ingeniería de software ágil y DevOps

Considerando cada uno de los objetivos y prácticas establecidas en la rúbrica, el equipo deberá reflexionar sobre los resultados obtenidos. Presentar ejemplos concretos de aplicación de prácticas, dificultades y soluciones encontradas.

4. Lecciones aprendidas

Resumir las lecciones aprendidas más relevantes del proyecto.